

DEEP LEARNING CON KERAS

Ignacio Gómez
2025

Frameworks para Redes Neuronales

Los frameworks para redes neuronales son herramientas esenciales en el campo del aprendizaje profundo (deep learning), ya que facilitan la creación, entrenamiento y evaluación de modelos de redes neuronales.

TensorFlow

•**Desarrollador:** Google Brain.

•**Lenguaje principal:** Python (también tiene soporte para C++, JavaScript, etc.).

•**Características:**

- Es uno de los frameworks más populares y completos.
- Ofrece flexibilidad para construir modelos desde cero o usar componentes predefinidos.
- Incluye TensorFlow Extended (TFX) para producción y TensorFlow Lite para dispositivos móviles.
- Soporta computación distribuida y aceleración por hardware (GPU/TPU).
- Integra Keras como su API de alto nivel.

PyTorch

Desarrollador: Facebook (ahora Meta).

Lenguaje principal: Python.

Características:

- Es muy popular en la comunidad de investigación debido a su flexibilidad y facilidad de uso.
- Utiliza tensores dinámicos (computación en tiempo de ejecución), lo que facilita la depuración.
- Integración con bibliotecas como torchvision, torchtext y torchaudio.
- Soporta computación distribuida y aceleración por hardware (GPU).

Otros: MXNet, Caffe, Theano, Fastai

Frameworks para Redes Neuronales

Los frameworks para redes neuronales son herramientas esenciales en el campo del aprendizaje profundo (deep learning), ya que facilitan la creación, entrenamiento y evaluación de modelos de redes neuronales.

Keras

Desarrollador: François Chollet (ahora parte de TensorFlow).

Lenguaje principal: Python.

Características:

- Es una API de alto nivel que simplifica la creación de redes neuronales.
- Originalmente era independiente, pero ahora es parte de TensorFlow (tf.keras).
- Ideal para principiantes debido a su simplicidad y legibilidad.
- Permite construir modelos de manera rápida y modular.

Keras 3

Soporte Multi-Backend: Keras 3 permite ejecutar workflows en **JAX**, **TensorFlow**, **PyTorch** y **OpenVINO** (solo para inferencia). Esto significa que puedes elegir el backend que mejor se adapte a tus necesidades y cambiar entre ellos sin modificar el código

API de Distribución para Entrenamiento a Gran Escala: Keras 3 introduce una API para manejar el paralelismo de datos y modelos a gran escala, especialmente con el backend de JAX. Esto facilita el entrenamiento de modelos en clusters de dispositivos, permitiendo configuraciones de sharding (fragmentación) para variables y tensores intermedios

Integración con Herramientas Externas:

- **Hugging Face Hub:** Keras 3 permite guardar y cargar modelos directamente desde Hugging Face Hub8.
- **Exportación a ONNX:** Ahora es posible exportar modelos de Keras a formato ONNX para su uso en diferentes entornos de inferencia

Frameworks para Redes Neuronales

Framework	Facilidad de uso	Flexibilidad	Rendimiento	Comunidad	Uso principal
TensorFlow	Media-Alta	Alta	Alto	Grande	Producción, investigación
Keras	Alta	Media	Medio	Grande	Prototipado, educación
PyTorch	Media	Alta	Alto	Grande	Investigación, producción
MXNet	Media	Alta	Alto	Mediana	Producción, nube
Caffe	Baja	Baja	Alto	Pequeña	Visión por computadora
Theano	Media	Media	Medio	Pequeña	Investigación (obsoleto)
JAX	Media	Alta	Alto	Creciente	Investigación avanzada
Fastai	Alta	Media	Medio	Mediana	Educación, prototipado

Creación de un modelo *Sequential*:

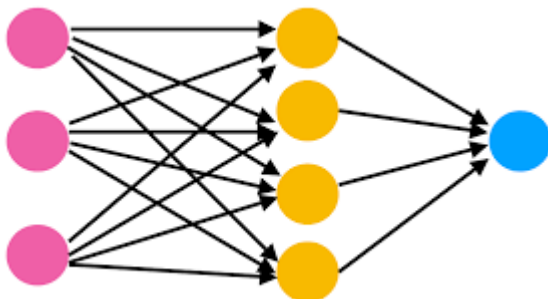
1. Definición de la arquitectura: `model = Sequential()`.
2. Compilación del modelo: `model.compile()`.
3. Entrenamiento del modelo: `model.fit()`.
4. Evaluación: `model.evaluate()`.
5. Predicción: `model.predict()`.
6. Resumen: `model.summary()`

Modelo *Sequential*: <https://keras.io/api/models/sequential/>

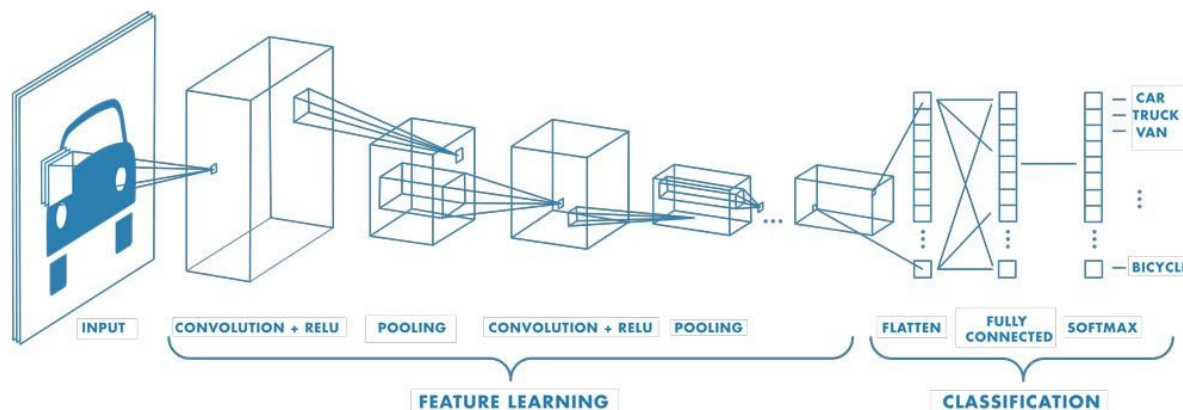
Es un contenedor lineal de capas. Ideal para modelos con flujo de datos de entrada a salida sin bifurcaciones.

```
model = Sequential([
    Flatten(input_shape=(28, 28)),      # Aplana las imágenes de 28x28 píxeles a un vector de 784 dimensiones
    Dense(300, activation='relu'),      # Capa densa con 300 neuronas y activación ReLU
    Dense(100, activation='relu'),      # Capa densa con 100 neuronas y activación ReLU
    Dense(10, activation='softmax')     # Capa de salida con 10 neuronas (una para cada clase) y activación softmax
])
```

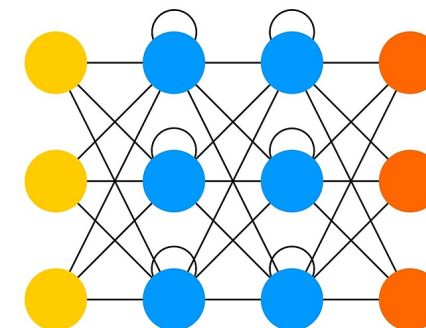
Redes completamente conectadas (densas)



Redes convolucionales (CNN)



Redes recurrentes (RNN).

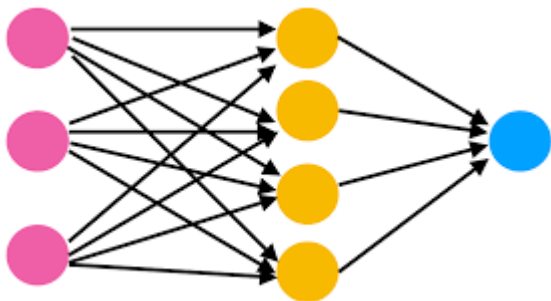


Modelo *Sequential*: <https://keras.io/api/models/sequential/>

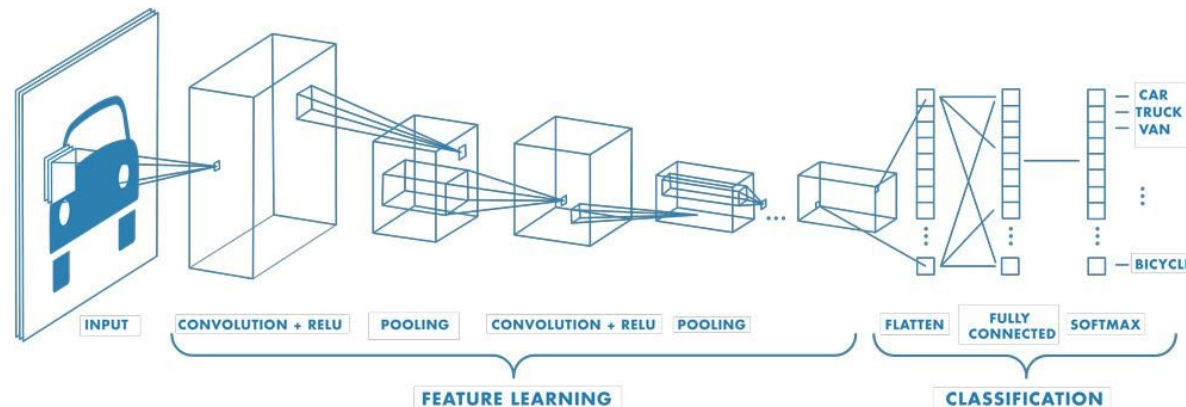
También se pueden añadir capas a un modelo existente.

```
model = models.Sequential()  
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))  
model.add(layers.MaxPooling2D((2, 2)))  
model.add(layers.Conv2D(64, (3, 3), activation='relu'))  
model.add(layers.MaxPooling2D((2, 2)))  
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

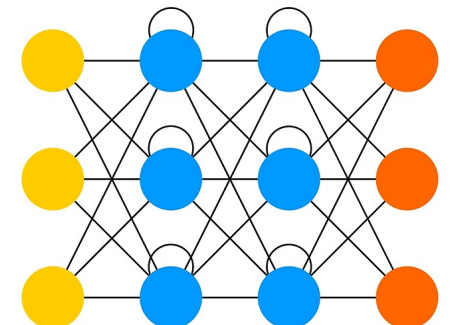
Redes completamente conectadas (densas)



Redes convolucionales (CNN)



Redes recurrentes (RNN).



Capas en una red neuronal:

- **Capas densas (Dense):** Totalmente conectadas, ideales para datos tabulares o partes finales de redes.

Parámetros clave: units, activation.

Ejemplo: `Dense(units = 100, activation='relu')`

- **Capas de Convolución (Conv2D):** Detectan patrones en datos espaciales (imágenes).

Parámetros clave: filters, kernel_size, padding, stride.

Ejemplo: `model.add(layers.Conv2D(64, (3, 3), padding = 'valid', activation='relu'))`

- **Capas de Pooling (MaxPooling2D):** Reducen la dimensionalidad espacial.

Parámetros clave: pool_size, stride.

Ejemplo: `model.add(layers.MaxPooling2D((2, 2)))`

- **Capas de regularización:**

- Dropout: previene el sobreajuste.
- BatchNormalization: acelera el entrenamiento y estabiliza la red.

Creación de un modelo *Sequential*:

1. Definición de la arquitectura: `Sequential()`.
2. Compilación del modelo: `model.compile()`.

```
model.compile(optimizer,  
              loss = None,  
              metrics = None,  
              loss_weights = None,  
              weighted_metrics = None,  
              run_eagerly = None,  
              steps_per_execution = None)
```

- Optimizador (`optimizer = 'adam', 'sgd', 'RMSprop'`): Especifica el algoritmo de optimización que ajusta los pesos del modelo durante el entrenamiento. Es obligatorio especificarlo. <https://keras.io/api/optimizers/>.

```
model.compile( optimizer=keras.optimizers.Adam(learning_rate=0.001),  
              loss='categorical_crossentropy',  
              metrics=['accuracy', tf.keras.metrics.Precision()] )
```

Creación de un modelo *Sequential*:

1. Definición de la arquitectura: `Sequential()`.
2. Compilación del modelo: `model.compile()`.

```
model.compile(optimizer,  
              loss = None,  
              metrics = None,  
              loss_weights = None,  
              weighted_metrics = None,  
              run_eagerly = None,  
              steps_per_execution = None)
```

- Función de pérdida (`loss`) : La función de pérdida evalúa el error en las predicciones. <https://keras.io/api/losses/>

Creación de un modelo *Sequential*:

- Función de pérdida (`loss`) : La función de pérdida evalúa el error en las predicciones.
<https://keras.io/api/losses/>
 - `binary_crossentropy`: Calcula la entropía cruzada entre las etiquetas verdaderas y las predicciones. Clasificación binaria
 - `categorical_crossentropy`: Para problemas multiclase donde las etiquetas están en formato one-hot encoding. Cada clase es representada por un vector binario con un "1" en la posición de la clase verdadera.
 - `sparse_categorical_crossentropy`: Se usa cuando las etiquetas están en formato entero (sin one-hot encoding). Esta pérdida aplica el formato correcto para interpretar las etiquetas enteras como categorías.
 - `mean_squared_error` (MSE): Compara la diferencia cuadrada promedio entre las predicciones y los valores verdaderos. Es común en tareas de regresión.
 - `mean_absolute_error` (MAE): Calcula la diferencia absoluta promedio entre las predicciones y los valores reales. Es útil cuando quieres minimizar el impacto de valores atípicos en la pérdida.

Creación de un modelo *Sequential*:

1. Definición de la arquitectura: `Sequential()`.
2. Compilación del modelo: `model.compile()`.

```
model.compile(optimizer,  
              loss = None,  
              metrics = None,  
              loss_weights = None,  
              weighted_metrics = None,  
              run_eagerly = None,  
              steps_per_execution = None)
```

- Métricas (`metrics='accuracy', 'mae'`): Lista de métricas que se calcularán y reportarán durante el entrenamiento y evaluación, además de la pérdida.

Ejemplo: `model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy', 'AUC'])`

Creación de un modelo *Sequential*:

1. Definición de la arquitectura: `Sequential()`.
2. Compilación del modelo: `model.compile()`.
3. Entrenamiento del modelo:

```
model.fit(x_train, y_train, epochs = 30, validation_data=(x_val, y_val))
```

- Datos de entrada: `(x_train, y_train)`
- Validación: `(validation_data=(x_val, y_val))`.
- Número de épocas (`epochs = 30`): Define el número de iteraciones completas sobre los datos.
- Tamaño del lote (`batch_size`): Con cuantos valores se calcula el gradiente y se actualizan los pesos.
- Porcentaje de datos en validación (`validation_split = 0.2`): Si no hemos definido `validation_data` podemos especificar el porcentaje de datos de entrenamiento que queremos usar para validar.
- Callbacks: Controlan dinámicamente el entrenamiento. Ejemplo más usado

Early_stopping: Detiene el entrenamiento si una métrica específica no mejora después de un número fijo de épocas.

```
early_stopping = EarlyStopping(monitor='val_loss', patience=5, restore_best_weights=True)
```

```
model.fit(train_images, train_labels, epochs=100, validation_split=0.2, callbacks=[early_stopping])
```

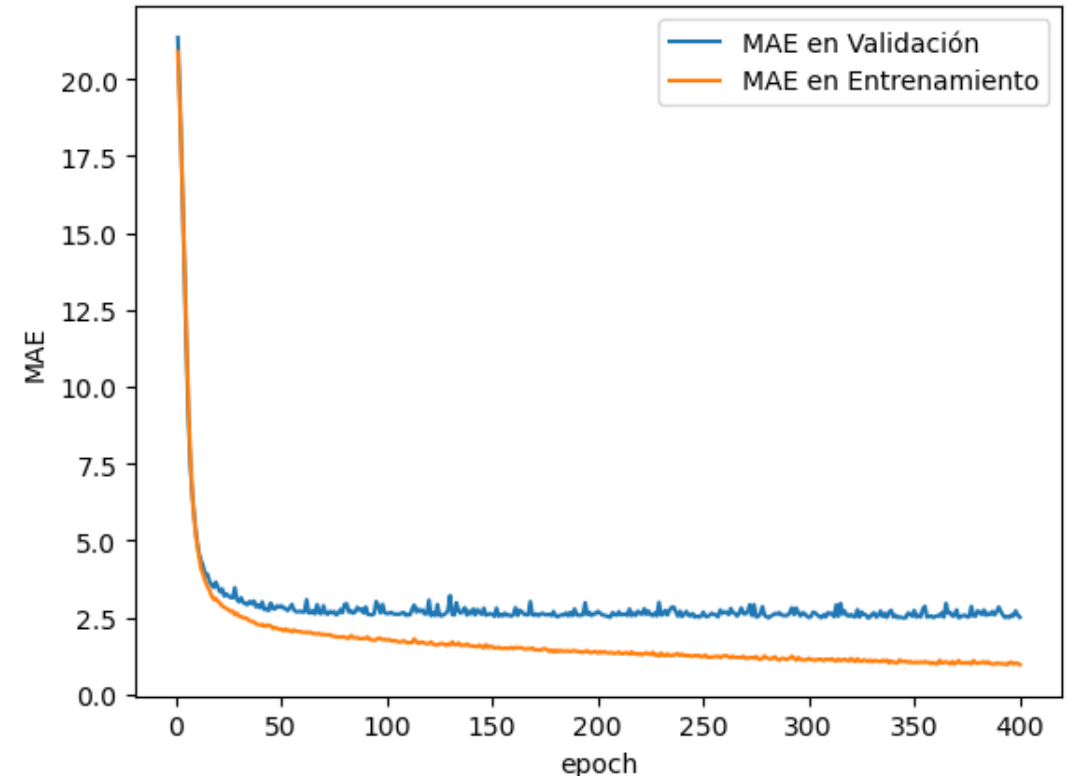
Creación de un modelo *Sequential*:

1. Definición de la arquitectura: `Sequential()`.
2. Compilación del modelo: `model.compile()`.
3. Entrenamiento del modelo:

`history = model.fit()` → Objeto de tipo diccionario con información del proceso de entrenamiento

Claves del diccionario:

- ➡ `loss`: El valor de la función de pérdida en los datos de entrenamiento para cada época.
- `accuracy` (o métricas equivalentes): La precisión en los datos de entrenamiento (si se especifica `metrics=["accuracy"]` al compilar el modelo).
- ➡ `val_loss`: El valor de la función de pérdida en los datos de validación (si se proporciona un conjunto de validación durante el entrenamiento).
- `val_accuracy` (o métricas equivalentes): La precisión en los datos de validación (si se incluye un conjunto de validación).



Creación de un modelo *Sequential*:

1. Definición de la arquitectura: `Sequential()`.
2. Compilación del modelo: `model.compile()`.
3. Entrenamiento del modelo: `model.fit()`.
4. Evaluación: `model.evaluate()`. Se utiliza para evaluar el rendimiento de un modelo entrenado en un conjunto de datos de prueba. Proporciona información sobre la pérdida (loss) y las métricas especificadas durante la compilación del modelo (`metrics=['accuracy']`).

```
# Evaluar el modelo
test_loss, test_acc = model.evaluate(X_test, y_test)
print("\n Pérdida en el conjunto de prueba:", test_loss)
print("\n Precisión en el conjunto de prueba:", test_acc)
```

Cuaderno: [Tutorial_MLP_CIFAR10_Keras_Regularizacion.ipynb](#)

Cuaderno: [Tutorial_MLP_Regresion_Keras_Regularizacion.ipynb](#)

Cuaderno: [Tutorial_MLP_Regresion_Aeroespacial_Keras.ipynb](#)

Teorema de la aproximación universal

Se dice que una red neuronal es un interpolador universal (o aproximador universal) debido a su capacidad para aproximar cualquier función continua con un grado arbitrario de precisión, siempre que cumpla ciertas condiciones. Este concepto se basa en el teorema de aproximación universal, que fue formalizado por George Cybenko en 1989 para redes neuronales con funciones de activación no lineales.

Teorema de aproximación universal:

El teorema establece que una red neuronal con:

- Una capa oculta (una sola capa oculta es suficiente).
- Un número suficiente de neuronas en esa capa oculta.
- Una función de activación no lineal (como la sigmoide, ReLU, tanh, etc.).

puede aproximar cualquier función continua definida en un espacio de entrada compacto (es decir, un conjunto cerrado y acotado) con un error arbitrariamente pequeño.

Cuaderno: `Interpolacion.ipynb`

Multilayer Feedforward Networks are Universal Approximators

KURT HORNIK

Technische Universität Wien

MAXWELL STINCHCOMBE AND HALBERT WHITE

University of California, San Diego

(Received 16 September 1988; revised and accepted 9 March 1989)

Sin embargo, en la práctica, la capacidad de la red debe equilibrarse con la complejidad del problema y la cantidad de datos disponibles para evitar el sobreajuste y garantizar un buen rendimiento.